

Events & Listeners

React to User Interactions Professionally

■■ EVENTS

■ LEARNING OUTCOME

By the end of this module you will handle all browser events, use the event object correctly, apply event delegation for dynamic UIs, implement debouncing and throttling, and use IntersectionObserver.

■ Skills You Will Gain

- Attach and remove event listeners
- Use event object: target, preventDefault, stopPropagation
- Understand event bubbling and capturing
- Apply event delegation for dynamic elements
- Use keyboard, mouse, form, and touch events
- Implement debounce and throttle

■ Events are how JavaScript reacts to the world. Without events, web pages are static brochures. With mastered events, they become powerful applications.

SECTION 1

addEventListener and Event Object

```
const btn = document.querySelector('#btn'); // Named function -- can be removed later function
handleClick(e) { console.log('Clicked!', e.target); } btn.addEventListener('click', handleClick);
btn.removeEventListener('click', handleClick); // same fn reference! // Options object
btn.addEventListener('click', handler, { once: true, // fire once then auto-remove passive: true,
// won't call preventDefault (improves scroll perf) capture: false, // bubble phase (default) });
// EVENT OBJECT properties document.addEventListener('click', function(event) { event.type //
'click' event.target // element that WAS clicked event.currentTarget // element listener is
ATTACHED to event.preventDefault() // stop default (form submit, link nav) event.stopPropagation()
// stop bubbling up DOM // Mouse: event.clientX, clientY, button, ctrlKey, shiftKey // Key:
event.key ('Enter','Escape','a'), event.code ('KeyA') }); // Form submit
document.querySelector('form').addEventListener('submit', (e) => { e.preventDefault(); // stop page
reload! const formData = new FormData(e.target); const name = formData.get('name'); const email =
formData.get('email'); console.log({ name, email }); }); // Keyboard shortcuts
document.addEventListener('keydown', (e) => { if (e.ctrlKey && e.key === 's') { e.preventDefault();
save(); } if (e.key === 'Escape') closeModal(); });
```

SECTION 2

Event Delegation & Debounce/Throttle

Event Bubbling -- Events travel UP the DOM tree

Clicking a button fires: button -> .card -> .grid -> body -> document EVENT DELEGATION: Attach ONE listener on the parent, check event.target. Benefits: works for dynamically added elements, fewer listeners = better performance.

```
// EVENT DELEGATION document.querySelector('.todo-list').addEventListener('click', (e) => { if
(e.target.matches('.delete-btn')) { const id = e.target.dataset.id; deleteTodo(id); } if
(e.target.matches('.complete-btn')) { e.target.closest('.todo-item').classList.toggle('completed');
} // Works for dynamically added items! }); // DEBOUNCE -- wait until user STOPS (search, resize,
validation) function debounce(fn, delay) { let timerId; return function(...args) {
clearTimeout(timerId); timerId = setTimeout(() => fn.apply(this, args), delay); }; } const search =
document.querySelector('#search'); search.addEventListener('input', debounce(e =>
fetchResults(e.target.value), 300)); // THROTTLE -- fire at most once per interval (scroll,
mousemove) function throttle(fn, limit) { let busy = false; return function(...args) { if (!busy) {
fn.apply(this, args); busy = true; setTimeout(() => { busy = false; }, limit); } }; }
window.addEventListener('scroll', throttle(() => updateProgress(), 100)); // INTERSECTION OBSERVER
-- lazy loading, scroll animations const observer = new IntersectionObserver((entries) => {
entries.forEach(entry => { if (entry.isIntersecting) { entry.target.classList.add('visible');
observer.unobserve(entry.target); // stop after first reveal } }); }, { threshold: 0.1 }); //
trigger at 10% visible document.querySelectorAll('.animate').forEach(el => observer.observe(el));
```

■ PRACTICE Q & A

Q1. What is event bubbling and how do you stop it?

A: When an event fires on an element, it propagates up through all ancestors. `event.stopPropagation()` stops it going further up. `event.stopImmediatePropagation()` also prevents other listeners on same element.

Q2. What is event delegation and why is it better?

A: Attaching ONE listener to a parent that handles events from all children by checking `event.target`. Better: one listener vs many, works for dynamically added elements, less memory usage.

Q3. What is the difference between `event.target` and `event.currentTarget`?

A: target: element that TRIGGERED the event (what was clicked). currentTarget: element the LISTENER is attached to. In delegation, currentTarget is parent; target is specific child clicked.

Q4. What is debouncing and when do you use it?

A: Delays execution until after a burst of events stops. Use for: search input (wait 300ms after typing stops), window resize handlers, form validation. Prevents hundreds of API calls.

Q5. What is the difference between mouseenter and mouseover?

A: mouseenter: fires only when mouse enters the element itself -- does NOT bubble. mouseover: fires when mouse enters element OR any descendant -- BUBBLES. Use mouseenter for simple hover effects.

■■ Events Cheat Sheet	
<code>el.addEventListener(type, fn)</code>	Attach event listener
<code>el.removeEventListener(type, fn)</code>	Remove -- same fn reference
<code>{ once: true }</code>	Auto-remove after first fire
<code>event.preventDefault()</code>	Stop default browser action
<code>event.stopPropagation()</code>	Stop bubbling up
<code>event.target</code>	Element that triggered
<code>event.currentTarget</code>	Element listener is on
<code>event.target.matches('.cls')</code>	Check what was clicked
<code>event.target.closest('.card')</code>	Find ancestor from target
<code>new FormData(formEl)</code>	Get all form field values
<code>debounce(fn, 300)</code>	Delay until user stops
<code>throttle(fn, 100)</code>	Max once per 100ms